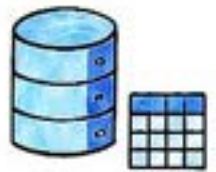


# SQL FUNDAMENTALS



## 1 What is SQL?

- SQL (Structured Query Language) is a standard language used to communicate with databases. It is used to store, retrieve, manage and manipulate data.

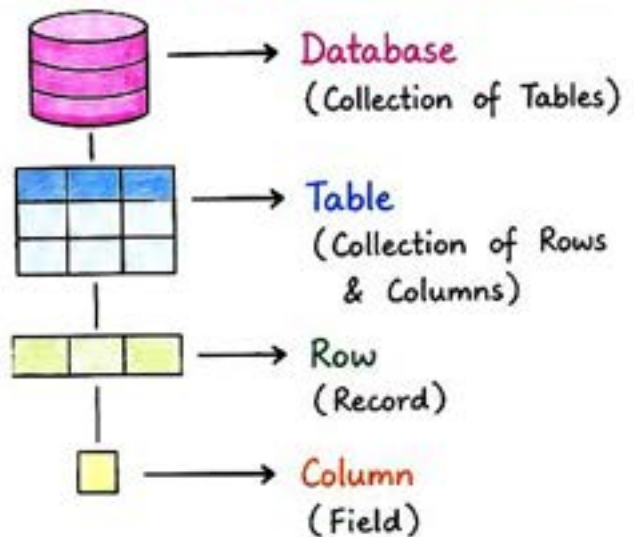
## 3 DBMS vs RDBMS

| DBMS                                                                                                                                                       | RDBMS                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Manages data in the form of files.</li> <li>No relationships between data.</li> <li>Example: File System</li> </ul> | <ul style="list-style-type: none"> <li>Manages data in tables (rows &amp; columns).</li> <li>Supports relationships using keys.</li> <li>Example: MySQL, Oracle, SQL Server</li> </ul> |

## 2 Why SQL?

- Easy to learn and use
- Works with all major databases
- Powerful for data analysis
- Widely used in IT industry
- Essential for Data Analysts, Developers, Testers, DBAs

## 4 Database, Table, Row & Column



## 5 SQL Syntax

- SQL commands follow a specific order.

```

SELECT column1, column2, ...
FROM table_name
WHERE condition;
  
```

★ SQL keywords are generally **NOT** case-sensitive. Best practice: write in UPPERCASE for clarity.

## 6 Data Types (Common)

- Numeric Types : INT, BIGINT, DECIMAL, FLOAT
- Character Types : CHAR, VARCHAR, TEXT
- Date & Time Types : DATE, TIME, DATETIME
- Boolean Type : BOOLEAN, BIT



SQL is the **foundation** of Data - Master the basics, and the advanced will be easy! ★



Follow **Kotha Nandakumari** for more content. Ajeet Yadav



# SQL NOTES

Slide 2

## 2. SQL COMMANDS



SQL commands are categorized into five main groups based on their functionality.

1

### DDL

(Data Definition Language)

Used to define or modify the database structure.

- **CREATE** - Creates database objects
- **ALTER** - Modifies existing objects
- **DROP** - Deletes objects
- **TRUNCATE** - Removes all rows from a table (structure remains)
- **RENAME** - Renames an object

Examples

```
CREATE TABLE Emp (...);  
ALTER TABLE Emp  
  ADD Column ...;  
DROP TABLE Emp;  
TRUNCATE TABLE Emp;  
RENAME TABLE Old_Emp  
  TO New_Emp;
```

2

### DML

(Data Manipulation Language)

Used to **insert, update, delete** and manage data inside tables.

- **INSERT** - Adds new data into a table
- **UPDATE** - Modifies existing data
- **DELETE** - Removes data from a table
- **MERGE (UPSERT)** - Inserts or updates data based on condition

Examples

```
INSERT INTO Emp VALUES (...);  
UPDATE Emp SET Salary = 50000  
  WHERE EmpID = 101;  
DELETE FROM Emp  
  WHERE EmpID = 101;  
MERGE INTO Emp ...;
```

3

### DQL

(Data Query Language)

Used to retrieve or query data from the database.

- **SELECT** - Retrieves data
- **FROM** - Specifies the table
- **WHERE** - Filters data
- **ORDER BY** - Sorts data
- **GROUP BY** - Groups data
- **HAVING** - Filters grouped data
- **DISTINCT** - Returns unique values
- **LIMIT / TOP** - Limits the number of rows

Examples

```
SELECT Name, Salary  
FROM Emp  
WHERE Dept = 'IT'  
ORDER BY Salary DESC  
LIMIT 10;
```

4

### DCL

(Data Control Language)

Used to control access and permissions on database objects.

- **GRANT** - Gives user privileges
- **REVOKE** - Removes user privileges

Examples

```
GRANT SELECT ON Emp  
  TO User1;  
REVOKE DELETE ON Emp  
  FROM User1;
```

5

### TCL

(Transaction Control Language)

Used to manage **transactions** in the database.

- **COMMIT** - Saves all changes permanently
- **ROLLBACK** - Undoes changes
- **SAVEPOINT** - Sets a point within a transaction to rollback

Examples

```
BEGIN TRANSACTION;  
UPDATE Emp SET Salary = 60000;  
SAVEPOINT SP1;  
ROLLBACK TO SP1;  
COMMIT;
```



# SQL NOTES

Slide 3

## 3. RETRIEVING DATA



Retrieving data is the **most common task** in SQL.

The SELECT statement is used to fetch data from one or more tables.

|                                                                     |                                                                                                                                             |                                                                                                                                     |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>1 SELECT</b><br>Used to select data from table(s).               | <ul style="list-style-type: none"><li>SELECT is used to retrieve data.</li><li>* (asterisk) selects all columns.</li></ul>                  | <b>Example</b><br>SELECT *<br>FROM Employees;                                                                                       |
| <b>2 DISTINCT</b><br>Used to return unique values.                  | <ul style="list-style-type: none"><li>Removes duplicate values from the result.</li><li>Useful when you want only unique entries.</li></ul> | <b>Example</b><br>SELECT DISTINCT Dept<br>FROM Employees;                                                                           |
| <b>3 WHERE</b><br>Used to filter records.                           | <ul style="list-style-type: none"><li>Filters rows based on a condition.</li><li>Returns only rows that satisfy the condition.</li></ul>    | <b>Example</b><br>SELECT Name, Salary<br>FROM Employees<br>WHERE Dept = 'IT';                                                       |
| <b>4 ORDER BY</b><br>Used to sort the result set.                   | <ul style="list-style-type: none"><li>Sorts data in ascending (ASC) or descending (DESC) order.</li><li>Default is ASC.</li></ul>           | <b>Example</b><br>SELECT Name, Salary<br>FROM Employees<br>ORDER BY Salary DESC;                                                    |
| <b>5 LIMIT / TOP</b><br>Used to limit the number of rows returned.  | <ul style="list-style-type: none"><li>LIMIT is used in MySQL, PostgreSQL, SQLite.</li><li>TOP is used in SQL Server.</li></ul>              | <b>Examples</b><br>-- MySQL<br>SELECT * FROM Employees LIMIT 5;<br>-- SQL Server<br>SELECT TOP 5 * FROM Employees;                  |
| <b>6 ALIAS</b><br>Used to give a temporary name to column or table. | <ul style="list-style-type: none"><li>Makes column or table names more readable.</li><li>Improves clarity of result set.</li></ul>          | <b>Examples</b><br>SELECT Name AS Emp_Name, Salary AS Pay<br>FROM Employees;<br><br>SELECT E.Name, E.Salary<br>FROM Employees AS E; |
| <b>7 COMMENTS</b><br>Used to add notes in SQL queries.              | <ul style="list-style-type: none"><li>Helps in understanding the query.</li><li>Ignored by the database engine.</li></ul>                   | <b>Examples</b><br>-- This is single line comment<br>SELECT * FROM Employees;<br>/* This is Multi-line comment */                   |

### Example

Employees

| EmpID | Name  | Dept    | Salary |
|-------|-------|---------|--------|
| 101   | Ravi  | IT      | 60000  |
| 102   | Sneha | HR      | 50000  |
| 103   | Amit  | Finance | 55000  |
| 104   | Neha  | IT      | 65000  |

SELECT Name, Dept FROM Employees;

→ Returns Name and Dept of all employees

SELECT DISTINCT Dept FROM Employees;

→ IT, HR, Finance

SELECT \* FROM Employees WHERE Dept = 'IT';

→ Rows where Dept is IT

SELECT \* FROM Employees ORDER BY Salary DESC;

→ Sorted by Salary (high to low)

SELECT \* FROM Employees LIMIT 2;

→ First 2 rows

SELECT Name AS Employee\_Name, Salary FROM Employees;

→ Column name as Employee\_Name

-- This is a comment

→ Ignored by SQL Engine



# SQL NOTES

Slide 4

## 4. FILTERING & OPERATORS



**Filtering** is used to extract only the required data from a table based on specific conditions.

|                                                                   |                                                                                                                                          |                                                                                                                                                  |
|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1 AND</b><br>Returns true if all conditions are true.          | <ul style="list-style-type: none"><li>Used to combine multiple conditions.</li><li>All conditions must be true for the result.</li></ul> | <b>Example</b><br><code>SELECT * FROM Employees<br/>WHERE Dept = 'IT' AND Salary &gt; 50000;</code>                                              |
| <b>2 OR</b><br>Returns true if any one of the conditions is true. | <ul style="list-style-type: none"><li>At least one condition must be true.</li><li>Increases result set.</li></ul>                       | <b>Example</b><br><code>SELECT * FROM Employees<br/>WHERE Dept = 'IT' OR Salary &gt; 50000;</code>                                               |
| <b>3 NOT</b><br>Reverses the result of a condition.               | <ul style="list-style-type: none"><li>Used to exclude data.</li><li>Often used with WHERE.</li></ul>                                     | <b>Example</b><br><code>SELECT * FROM Employees<br/>WHERE NOT Dept = 'HR';</code>                                                                |
| <b>4 BETWEEN</b><br>Selects values within a given range.          | <ul style="list-style-type: none"><li>Inclusive of both start and end values.</li><li>Works with numbers, dates and text.</li></ul>      | <b>Example</b><br><code>SELECT * FROM Employees<br/>WHERE Salary BETWEEN 40000 AND 70000;</code>                                                 |
| <b>5 IN</b><br>Checks if a value matches any value in a list.     | <ul style="list-style-type: none"><li>Simpler way to write multiple OR conditions.</li><li>Supports subquery too.</li></ul>              | <b>Example</b><br><code>SELECT * FROM Employees<br/>WHERE Dept IN ('IT', 'HR', 'Finance');</code>                                                |
| <b>6 LIKE</b><br>Searches for a pattern in text values.           | <ul style="list-style-type: none"><li>Uses wildcards.</li><li>% → any number of characters<br/>_ → single character</li></ul>            | <b>Example</b><br><code>SELECT * FROM Employees<br/>WHERE Name LIKE 'A%'; -- Starts with A<br/>WHERE Name LIKE '%an%'; -- Contains 'an'</code>   |
| <b>7 IS NULL</b><br>Checks for missing or blank values.           | <ul style="list-style-type: none"><li>NULL means no value or unknown.</li><li>Use IS NOT NULL to check available values.</li></ul>       | <b>Example</b><br><code>SELECT * FROM Employees<br/>WHERE ManagerID IS NULL;<br/>SELECT * FROM Employees<br/>WHERE ManagerID IS NOT NULL;</code> |

### EXAMPLE TABLE

#### Employees

| EmpID | Name  | Dept    | Salary | JoinDate   |
|-------|-------|---------|--------|------------|
| 101   | Ravi  | IT      | 60000  | 2023-01-15 |
| 102   | Sneha | HR      | 50000  | 2022-03-10 |
| 103   | Amit  | Finance | 55000  | 2021-07-22 |
| 104   | Neha  | IT      | NULL   | 2023-05-18 |
| 105   | Kiran | HR      | 45000  | 2022-11-30 |

### More Examples

- `SELECT * FROM Employees  
WHERE Dept = 'IT' AND Salary > 50000;`
- `SELECT * FROM Employees  
WHERE Dept = 'IT' OR Salary > 50000;`
- `SELECT * FROM Employees  
WHERE Salary BETWEEN 40000 AND 60000;`
- `SELECT * FROM Employees  
WHERE Name LIKE 'S%';`
- `SELECT * FROM Employees  
WHERE Dept IN ('HR', 'Finance');`
- `SELECT * FROM Employees  
WHERE ManagerID IS NULL;`

### QUICK TIPS

- ★ Use AND to narrow down results.
- ★ Use OR to widen results.
- ★ Use IN for multiple exact matches.
- ★ Use LIKE for text pattern search.
- ★ Always handle NULL values

Ajeet Yadav



## 5. AGGREGATE FUNCTIONS



Aggregate functions perform calculations on a set of values and return a single value.

### 1 COUNT()

- Returns the number of rows.
- COUNT(column) counts only non-NULL values.

Example

```
SELECT COUNT(*)  
FROM Employees;
```

### 2 SUM()

- Returns the total sum.
- Works only on numeric columns.

Example

```
SELECT SUM(Salary)  
FROM Employees;
```

### 3 AVG()

- Returns the average value.
- Works only on numeric columns.

Example

```
SELECT AVG(Salary)  
FROM Employees;
```

### 4 MIN()

- Returns the minimum value.
- Works on numbers, dates and text.

Example

```
SELECT MIN(Salary)  
FROM Employees;
```

### 5 MAX()

- Returns the maximum value.
- Works on numbers, dates and text.

Example

```
SELECT MAX(Salary)  
FROM Employees;
```

### 6 GROUP BY

- Groups rows that have the same values.
- Used with aggregate functions.

Example

```
SELECT Dept, COUNT(*) AS Emp_Count,  
       AVG(Salary) AS Avg_Salary  
FROM Employees  
GROUP BY Dept;
```

Result

| Dept    | Emp_Count | Avg_Salary |
|---------|-----------|------------|
| Finance | 1         | 65000      |
| HR      | 1         | 50000      |
| IT      | 3         | 53333      |

### 7 HAVING

- Filters groups after GROUP BY.
- WHERE cannot be used with aggregate functions.

Example

```
SELECT Dept, COUNT(*) AS Emp_Count  
FROM Employees  
GROUP BY Dept  
HAVING COUNT(*) > 1;
```

Result

| Dept | Emp_Count |
|------|-----------|
| IT   | 3         |



### Key Points

- ✓ Aggregate functions ignore NULL values (except COUNT(\*)).
- ✓ Can be used with GROUP BY for group-wise calculations.
- ✓ Can be filtered using HAVING (not WHERE).
- ✓ Commonly used in reports, dashboards and analytics.

### Sample Data

#### Employees

| EmpID | Name  | Dept    | Salary |
|-------|-------|---------|--------|
| 101   | Ravi  | IT      | 60000  |
| 102   | Sneha | HR      | 50000  |
| 103   | Amit  | IT      | 55000  |
| 104   | Neha  | Finance | 65000  |
| 105   | Kiran | IT      | 45000  |

### More Examples

- ✓ Total salary of IT dept:  
SELECT SUM(Salary) FROM Employees WHERE Dept = 'IT';
- ✓ Highest salary:  
SELECT MAX(Salary) FROM Employees;
- ✓ Number of employees in Finance:  
SELECT COUNT(\*) FROM Employees WHERE Dept = 'Finance';

### Quick Tips

- ★ Use COUNT(\*) to count all rows including NULLs.
- ★ Use GROUP BY for department, product, customer wise analysis.
- ★ Use HAVING to filter aggregated results.
- ★ Combine multiple aggregate functions for better insights.
- ★ Essential for interview questions and real-world reporting.



# SQL NOTES

Slide 6

## 6. SQL JOINS



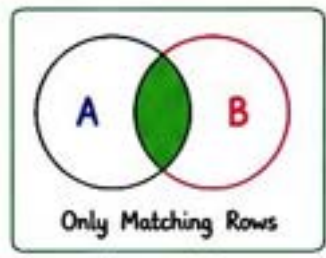
JOINS are used to combine rows from two or more tables based on a related column between them.

### 1 INNER JOIN

- Returns only the matching rows from both tables.
- Non-matching rows are excluded.

Example

```
SELECT A.col1, B.col2
FROM TableA A
INNER JOIN TableB B
ON A.id = B.id;
```

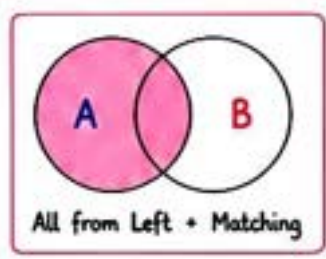


### 2 LEFT JOIN

- Returns all rows from the left table and matching rows from the right table.
- Non-matching rows from right table will be NULL.

Example

```
SELECT A.col1, B.col2
FROM TableA A
LEFT JOIN TableB B
ON A.id = B.id;
```

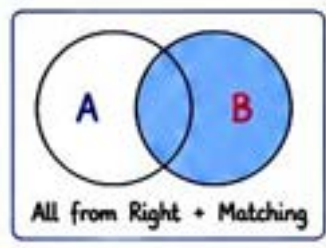


### 3 RIGHT JOIN

- Returns all rows from the right table and matching rows from the left table.
- Non-matching rows from left table will be NULL.

Example

```
SELECT A.col1, B.col2
FROM TableA A
RIGHT JOIN TableB B
ON A.id = B.id;
```

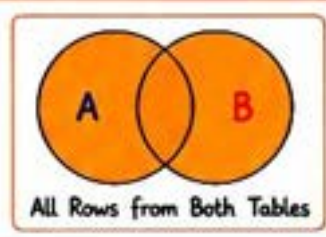


### 4 FULL OUTER JOIN

- Returns all rows when there is a match in either left or right table.
- Non-matching rows from both tables are included as NULL.

Example

```
SELECT A.col1, B.col2
FROM TableA A
FULL OUTER JOIN TableB B
ON A.id = B.id;
```



### 5 CROSS JOIN

- Returns the Cartesian product of both tables.
- Every row from left table is combined with every row from right table.

Example

```
SELECT A.col1, B.col2
FROM TableA A
CROSS JOIN TableB B;
```

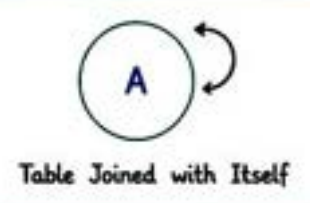


### 6 SELF JOIN

- A table is joined with itself.
- Useful for hierarchical or recursive data.

Example

```
SELECT A.name AS Emp, B.name AS Manager
FROM Employees A
LEFT JOIN Employees B
ON A.manager_id = B.id;
```



#### SAMPLE DATA

Employees (A)

| EmpID | Name  | DeptID |
|-------|-------|--------|
| 1     | Ravi  | 101    |
| 2     | Sneha | 102    |
| 3     | Amit  | 103    |
| 4     | Neha  | 104    |

Departments (B)

| DeptID | DeptName  |
|--------|-----------|
| 101    | IT        |
| 102    | HR        |
| 103    | Finance   |
| 105    | Marketing |

#### EXAMPLE - INNER JOIN

```
SELECT A.Name AS DeptName
FROM Employees A
INNER JOIN Departments B
ON A.DeptID = B.DeptID;
```

Result

| Name  | DeptName |
|-------|----------|
| Ravi  | IT       |
| Sneha | HR       |
| Amit  | Finance  |

Ajeet Yadav



# SQL NOTES

Slide 7

## 7. CONSTRAINTS & KEYS



Constraints are rules applied to columns or tables to ensure data accuracy and reliability.

### 1 PRIMARY KEY

- Uniquely identifies each row in a table.
- Cannot contain NULL values.

Example

```
CREATE TABLE Students (  
  StudentID INT PRIMARY KEY,  
  Name VARCHAR(50)  
);
```

### Key Points

- ✓ A table can have only one PRIMARY KEY.
- ✓ Automatically creates a unique index.
- ✓ Used to establish entity integrity.

### 2 FOREIGN KEY

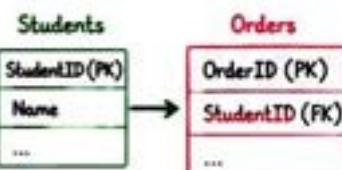
- Establishes a link between two tables.
- Enforces referential integrity.

Example

```
CREATE TABLE Orders (  
  OrderID INT PRIMARY KEY,  
  StudentID INT,  
  FOREIGN KEY (StudentID)  
  REFERENCES Students(StudentID)  
);
```

### Referential Integrity

A Foreign Key value must either exist in the referenced table or be NULL.



### 3 UNIQUE KEY

- Ensures all values in a column (or set of columns) are unique.
- Can contain one NULL value (in most DBMS).

Example

```
CREATE TABLE Users (  
  UserID INT PRIMARY KEY,  
  Email VARCHAR(100) UNIQUE  
);
```

### 4 NOT NULL

- Ensures a column cannot have NULL values.

Example

```
CREATE TABLE Employees (  
  EmpID INT PRIMARY KEY,  
  Name VARCHAR(50) NOT NULL  
);
```

### Quick Example

```
CREATE TABLE Departments (  
  DeptID INT PRIMARY KEY,  
  DeptName VARCHAR(50) NOT NULL,  
  Location VARCHAR(50) UNIQUE  
);
```

### 5 CHECK CONSTRAINT

- Ensures values in a column satisfy a specific condition.

Example

```
CREATE TABLE Products (  
  ProductID INT PRIMARY KEY,  
  Price DECIMAL(10,2) CHECK (Price >= 0)  
);
```

### 6 DEFAULT CONSTRAINT

- Provides a default value for a column when no value is specified.

Example

```
CREATE TABLE Employees (  
  EmpID INT PRIMARY KEY,  
  Status VARCHAR(20) DEFAULT 'Active'  
);
```

### Default Values

- Reduces manual effort.
- Ensures consistent data.
- Can be combined with NOT NULL.

### Combined Example

```
CREATE TABLE Courses (  
  CourseID INT PRIMARY KEY,  
  CourseName VARCHAR(100) NOT NULL,  
  Credits INT CHECK (Credits > 0),  
  DeptID INT,  
  UNIQUE (CourseName),  
  FOREIGN KEY (DeptID) REFERENCES Departments(DeptID),  
  Status VARCHAR(20) DEFAULT 'Active'  
);
```

Departments

| DeptID | DeptName |
|--------|----------|
| 1      | IT       |
| 2      | HR       |
| 3      | Finance  |

Courses

| CourseID | CourseName | Credits | DeptID | Status |
|----------|------------|---------|--------|--------|
| 101      | SQL Basics | 3       | 1      | Active |
| 102      | Python     | 4       | 1      | Active |
| 103      | DBMS       | 3       | 2      | Active |



**Remember:** Constraints help maintain data quality, accuracy and consistency in the database.

Ajeet Yadav



# SQL NOTES

Slide 8

## 8. SQL FUNCTIONS



SQL Functions are pre-defined formulas that perform operations on data and return a value.

### 1 STRING FUNCTIONS

- **UPPER()** - Converts to upper case
- **LOWER()** - Converts to lower case
- **LEN()** - Returns length of string
- **SUBSTRING(str, start, length)** - Extracts part of string
- **REPLACE(str, old, new)** - Replaces substring
- **CONCAT(str1, str2)** - Joins strings
- **TRIM(str)** - Removes leading and trailing spaces

Example

```
SELECT UPPER('sql notes');  
-- SQL NOTES  
SELECT LEN('database');  
-- 8  
SELECT SUBSTRING('SQL Notes', 1, 3); -- SQL  
SELECT REPLACE('SQL is fun', 'fun', 'easy');  
-- SQL is easy  
SELECT CONCAT('SQL', ' Notes'); -- SQL Notes  
SELECT TRIM(' Hello ');  
-- Hello
```

### 2 NUMERIC FUNCTIONS

- **ABS(n)** - Absolute value
- **ROUND(n, d)** - Round to d decimal places
- **CEILING(n)** - Smallest integer greater than or equal to n
- **FLOOR(n)** - Largest integer less than or equal to n
- **POWER(n, p)** - n raised to the power p
- **MOD(n, m)** - Remainder of n divided by m

Example

```
SELECT ABS(-25);  
-- 25  
SELECT ROUND(25.567,2);  
-- 25.57  
SELECT CEILING(25.1);  
-- 26  
SELECT FLOOR(25.9);  
-- 25  
SELECT POWER(2,3);  
-- 8  
SELECT MOD(10,3);  
-- 1
```

### 3 DATE FUNCTIONS

- **NOW() / CURRENT\_TIMESTAMP** - Current date & time
- **CURDATE()** - Current date
- **CURTIME()** - Current time
- **DATE\_ADD(date, INTERVAL n unit)** - Add time interval
- **DATEDIFF(date1, date2)** - Difference between dates
- **EXTRACT(unit FROM date)** - Extract part of date

Example

```
SELECT NOW();  
-- 2025-05-24 10:30:45  
SELECT CURDATE();  
-- 2025-05-24  
SELECT DATE_ADD('2025-05-24',  
INTERVAL 7 DAY);  
-- 2025-05-31  
SELECT DATEDIFF('2025-05-24',  
'2025-05-01'); -- 23  
SELECT EXTRACT(YEAR FROM  
'2025-05-24'); -- 2025
```

### 4 CONVERSION FUNCTIONS

- **CAST(value AS type)** - Converts value to data type
- **CONVERT(value, type)** - Converts value to data type

Example

```
SELECT CAST('123' AS INT); -- 123  
SELECT CAST('2025-05-24' AS DATE);  
-- 2025-05-24  
SELECT CONVERT(123.456, CHAR);  
-- '123.456'
```

### 5 CASE STATEMENT

Used for conditional logic in SELECT, UPDATE etc.

Example

```
SELECT Name, Salary,  
CASE  
WHEN Salary >= 60000 THEN 'High'  
WHEN Salary >= 40000 THEN 'Medium'  
ELSE 'Low'  
END AS Salary_Level  
FROM Employees;
```

| Name  | Salary | Salary_Level |
|-------|--------|--------------|
| Ravi  | 70000  | High         |
| Sneha | 50000  | Medium       |
| Amit  | 30000  | Low          |

### 6 NULL HANDLING FUNCTIONS

- **IFNULL(expr, value)** - Returns value if expr is NULL
- **COALESCE(expr1, expr2, ...)** - Returns first non-NULL expression
- **NULLIF(expr1, expr2)** - Returns NULL if expr1 = expr2, else expr1

Example

```
SELECT IFNULL(NULL, 'N/A'); -- N/A  
SELECT COALESCE(NULL, NULL, 'Data'); -- Data  
SELECT NULLIF(10,10); -- NULL  
SELECT NULLIF(10,20); -- 10
```



Quick Examples (All in One)

```
SELECT UPPER('hello'); -- HELLO  
SELECT ROUND(123.456, 1); -- 123.5  
SELECT CURDATE(); -- 2025-05-24  
SELECT CAST('99.9' AS DECIMAL(5,1)); -- 99.9
```

```
SELECT CASE WHEN 5 > 3 THEN 'Yes' ELSE 'No' END; -- Yes  
SELECT IFNULL(NULL, 'Value'); -- Value  
SELECT COALESCE(NULL, 'A', 'B'); -- A  
SELECT NULLIF('SQL', 'SQL'); -- NULL
```

Key Takeaway

Functions make queries powerful, readable and efficient. Mastering them is essential for real-world work.

Ajeet Yadav



# SQL NOTES

Slide 9

## 9. SUBQUERIES & NESTED QUERIES



A subquery is a query inside another query. Used to solve complex problems by breaking them into smaller parts.

### 1 WHAT IS A SUBQUERY?

- A query within another SQL query.
- Can be used in SELECT, FROM, WHERE, HAVING clauses.
- Returns a single value, multiple values, or a table.

Example

```
SELECT Name
FROM Employees
WHERE DeptID = (SELECT DeptID
                FROM Departments
                WHERE DeptName = 'IT');
```

### Key Points

- ✓ Improves readability.
- ✓ Helps in step-by-step problem solving.
- ✓ Can return single value, multiple values or tables.
- ✓ Used for filtering, comparison and calculations.

### 2 TYPES OF SUBQUERIES

- **Single Row Subquery** → Returns exactly one row.
- **Multiple Row Subquery** → Returns more than one row.
- **Multiple Column Subquery** → Returns more than one column.
- **Correlated Subquery** → Refers to columns from outer query.

Example (Multiple Row Subquery)

```
SELECT Name
FROM Employees
WHERE DeptID IN (SELECT DeptID
                FROM Departments
                WHERE Location = 'Bangalore');
```

### Common Operators

- = Equal to
- > Greater than
- < Less than
- >= Greater than or equal
- <= Less than or equal
- <> Not equal to
- IN In a list of values
- NOT IN Not in a list of values
- EXISTS Exists
- NOT EXISTS Does not exist

### 3 SUBQUERY WITH COMPARISON OPERATORS

- Use operators like =, >, <, >=, <=, <>
- Can be used with single row subqueries.

Example

```
SELECT Name, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary)
                FROM Employees);
```

### 4 SUBQUERY WITH EXISTS / NOT EXISTS

- **EXISTS** returns TRUE if subquery returns any rows.
- **NOT EXISTS** returns TRUE if subquery returns no rows.

Example

```
SELECT Name
FROM Employees e
WHERE EXISTS (SELECT 1
              FROM Projects p
              WHERE p.EmpID = e.EmpID);
```

### 5 CORRELATED SUBQUERY

- Depends on the outer query.
- Evaluated once for each row processed by the outer query.
- Usually slower than other subqueries.

Example

```
SELECT e.Name
FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary)
                  FROM Employees
                  WHERE DeptID = e.DeptID);
```

### When to Use?

- ✓ When complex joins are needed.
- ✓ When comparisons depend on row-by-row data.
- ✓ When EXISTS / NOT EXISTS logic is required.

### Quick Examples

- Employees earning more than company average:  

```
SELECT * FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```
- Employees not in IT department:  

```
SELECT * FROM Employees
WHERE DeptID NOT IN (SELECT DeptID FROM Departments
                    WHERE DeptName = 'IT');
```
- Departments with no employees:  

```
SELECT * FROM Departments d
WHERE NOT EXISTS (SELECT 1 FROM Employees e
                  WHERE e.DeptID = d.DeptID);
```

### Best Practices

- ✓ Use subqueries for clarity, not just for shorter code.
- ✓ Prefer JOINS over subqueries for better performance, if possible.
- ✓ Use EXISTS instead of IN for better performance on large datasets.
- ✓ Always test subqueries separately before using them in main query.
- ✓ Avoid using subqueries that return multiple rows with single value operators.



# SQL NOTES

Slide 10

## 10. AGGREGATE FUNCTIONS & GROUP BY



Aggregate functions perform calculations on sets of rows and return a single value.

### 1 AGGREGATE FUNCTIONS

- Used to perform calculations on multiple rows.
- Ignore NULL values (except COUNT(\*)).
- Return a single result.

| Function      | Description                        |
|---------------|------------------------------------|
| COUNT(*)      | Counts total number of rows        |
| COUNT(column) | Counts non-NULL values in a column |
| SUM(column)   | Returns sum of values              |
| AVG(column)   | Returns average of values          |
| MIN(column)   | Returns minimum value              |
| MAX(column)   | Returns maximum value              |

#### Example

```
SELECT COUNT(*) AS Total_Employees,  
       AVG(Salary) AS Avg_Salary,  
       MIN(Salary) AS Min_Salary,  
       MAX(Salary) AS Max_Salary,  
       SUM(Salary) AS Total_Salary  
FROM Employees;
```

### 2 GROUP BY CLAUSE

- Groups rows that have the same values in specified columns.
- Used with aggregate functions to get summary for each group.
- All columns in SELECT (except aggregates) must be in GROUP BY.

#### Example

```
SELECT Department, COUNT(*) AS Total_Employees,  
       AVG(Salary) AS Avg_Salary  
FROM Employees  
GROUP BY Department  
ORDER BY Total_Employees DESC;
```

Employees Table

| EmpID | Name  | Dept    | Salary |
|-------|-------|---------|--------|
| 1     | Ravi  | IT      | 60000  |
| 2     | Sneha | IT      | 50000  |
| 3     | Amit  | HR      | 40000  |
| 4     | Neha  | HR      | 45000  |
| 5     | Karan | Finance | 55000  |



Grouped by Dept

| Dept    | Total_Employees | Avg_Salary |
|---------|-----------------|------------|
| IT      | 2               | 55000.00   |
| HR      | 2               | 42500.00   |
| Finance | 1               | 55000.00   |

### 3 HAVING CLAUSE

- Used to filter groups after GROUP BY.
- WHERE filters rows before grouping, HAVING filters groups after grouping.

#### Example

```
SELECT Department, COUNT(*) AS Total_Employees,  
       AVG(Salary) AS Avg_Salary  
FROM Employees  
GROUP BY Department  
HAVING COUNT(*) > 1  
ORDER BY Total_Employees DESC;
```

### 4 WHERE vs HAVING

| WHERE                                       | HAVING                          |
|---------------------------------------------|---------------------------------|
| Filters individual rows                     | Filters groups                  |
| Used before grouping                        | Used after grouping             |
| Cannot use aggregate functions              | Can use aggregate functions     |
| Improves performance by reducing rows early | Works on the grouped result set |

#### Example

```
SELECT Department, COUNT(*) AS Total_Employees,  
       AVG(Salary) AS Avg_Salary  
FROM Employees  
WHERE Salary > 40000 -- filters rows first  
GROUP BY Department  
HAVING AVG(Salary) > 45000 -- filters groups  
ORDER BY Avg_Salary DESC;
```

### 5 TIPS



- ✓ COUNT(\*) counts all rows including NULLs.
- ✓ COUNT(column) counts only non-NULL values.
- ✓ Always use meaningful aliases for readability.
- ✓ Use ORDER BY to sort the final result.
- ✓ Combine WHERE and HAVING for better control.

### 6 OTHER EXAMPLES



- Total salary paid in each department:  

```
SELECT Department, SUM(Salary) AS Total_Salary  
FROM Employees  
GROUP BY Department;
```
- Departments with more than 3 employees:  

```
SELECT Department, COUNT(*) AS Total_Employees  
FROM Employees  
GROUP BY Department  
HAVING COUNT(*) > 3;
```



# SQL NOTES

Slide 11

## 11. ORDER BY, LIMIT & SET OPERATIONS



These clauses and operations help to sort, restrict and combine result sets efficiently.

### 1 ORDER BY CLAUSE

- Used to sort the result set.
- ASC → Ascending order (default)
- DESC → Descending order
- Can sort by one or more columns.

#### Example

```
-- Employees table
SELECT EmpID, Name, Salary
FROM Employees
ORDER BY Salary DESC; -- High to Low

SELECT EmpID, Name, Dept
FROM Employees
ORDER BY Dept ASC, Name ASC;
```

#### Key Points



- ✓ ORDER BY is always the last clause.
- ✓ You can sort by column name or position.
- ✓ NULL values come first in ASC and last in DESC (varies by DBMS).

### 2 LIMIT / TOP CLAUSE

- Used to limit the number of rows returned.
- Syntax may vary by DBMS.
- Commonly used for pagination or top-N results.

#### Examples

```
-- MySQL / PostgreSQL
SELECT * FROM Employees
ORDER BY Salary DESC
LIMIT 5; -- Top 5 highest salary

-- SQL Server
SELECT TOP 5 * FROM Employees
ORDER BY Salary DESC;
```

#### Use Cases



- ✓ Display top N records
- ✓ Pagination (LIMIT with OFFSET)
- ✓ Improve performance by restricting data

### 3 SET OPERATIONS

- Used to combine the result sets of two or more SELECT statements.
- The number and data type of columns must be the same in all queries.

| Operation      | Description                                                         | Example                                                                                                        | Notes                                                                                                 |
|----------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| UNION          | Combines results of two queries and removes duplicates.             | SELECT Name FROM Employees<br>UNION<br>SELECT Name FROM Contractors;                                           | <ul style="list-style-type: none"><li>• Removes duplicate rows.</li><li>• Default behavior.</li></ul> |
| UNION ALL      | Combines results and keeps all duplicates.                          | SELECT Name FROM Employees<br>UNION ALL<br>SELECT Name FROM Contractors;                                       | <ul style="list-style-type: none"><li>• Faster than UNION.</li><li>• Includes duplicates.</li></ul>   |
| INTERSECT      | Returns only the common rows from both queries.                     | SELECT Name FROM Employees<br>INTERSECT<br>SELECT Name FROM Contractors;                                       | <ul style="list-style-type: none"><li>• Returns rows present in both result sets.</li></ul>           |
| EXCEPT / MINUS | Returns rows from the first query that are not in the second query. | SELECT Name FROM Employees<br>EXCEPT<br>SELECT Name FROM Contractors;<br>-- Oracle use MINUS instead of EXCEPT | <ul style="list-style-type: none"><li>• Shows only unique rows from first query.</li></ul>            |

#### Set Operations Example

```
-- Employees in either table (without duplicates)
SELECT EmpID FROM Employees
UNION
SELECT EmpID FROM Contractors;

-- Employees who are not contractors
SELECT EmpID FROM Employees
EXCEPT
SELECT EmpID FROM Contractors;
```

#### Quick Tips



- ✓ ORDER BY sorts the data.
- ✓ LIMIT restricts the rows.
- ✓ UNION removes duplicates.
- ✓ UNION ALL keeps duplicates.
- ✓ INTERSECT → common rows.
- ✓ EXCEPT → unique rows from first query.

#### Practice Query

```
-- Top 3 highest salary from all employees and contractors
(SELECT Name, Salary FROM Employees
UNION ALL
SELECT Name, Salary FROM Contractors)
ORDER BY Salary DESC
LIMIT 3;
```



# SQL NOTES

Slide 12

## 12. JOINS & SUBQUERIES IN DETAIL



Joins combine rows from two or more tables based on a related column.

### 1 TYPES OF JOINS

- **INNER JOIN** → Returns matching rows from both tables.
- **LEFT JOIN** → All rows from left table and matching from right.
- **RIGHT JOIN** → All rows from right table and matching from left.
- **FULL OUTER JOIN** → All rows from both tables.
- **CROSS JOIN** → Cartesian product of both tables.

#### Examples

```
-- INNER JOIN
SELECT e.EmpID, e.Name, d.DeptName
FROM Employees e
INNER JOIN Departments d ON e.DeptID = d.DeptID;

-- LEFT JOIN
SELECT e.EmpID, e.Name, d.DeptName
FROM Employees e
LEFT JOIN Departments d ON e.DeptID = d.DeptID;

-- RIGHT JOIN
SELECT e.EmpID, e.Name, d.DeptName
FROM Employees e
RIGHT JOIN Departments d ON e.DeptID = d.DeptID;

-- FULL OUTER JOIN (syntax may vary by DBMS)
SELECT e.EmpID, e.Name, d.DeptName
FROM Employees e
FULL OUTER JOIN Departments d ON e.DeptID = d.DeptID;

-- CROSS JOIN
SELECT e.Name, d.DeptName
FROM Employees e
CROSS JOIN Departments d;
```

#### Key Points



- ✓ **INNER JOIN** returns only matching rows.
- ✓ **LEFT / RIGHT JOIN** returns all rows from one side.
- ✓ **FULL OUTER JOIN** returns all rows from both sides.
- ✓ Use table aliases (e, d) for better readability.
- ✓ Always use meaningful column names in joins.

### 2 SUBQUERIES

- A subquery is a query inside another query.
- Can be used in SELECT, FROM, WHERE, HAVING clauses.
- Helps in solving complex problems in smaller steps.

#### Types of Subqueries

- **Single Row Subquery** → Returns one row, one column.
- **Multiple Row Subquery** → Returns multiple rows.
- **Multiple Column Subquery** → Returns multiple columns.
- **Correlated Subquery** → Depends on the outer query.
- **Non-Correlated Subquery** → Independent of outer query.

#### Example (Single Row Subquery)

```
-- Employees earning more than average salary
SELECT Name, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary)
FROM Employees);
```

### 3 SUBQUERY EXAMPLES

| Use Case                    | Example                                                                                                                  | Description                                                                             |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| IN Operator                 | <pre>SELECT Name FROM Employees WHERE DeptID IN (SELECT DeptID FROM Departments WHERE Location = 'Bangalore');</pre>     | Returns employees whose DeptID exists in the Departments table with location Bangalore. |
| EXISTS Operator             | <pre>SELECT Name FROM Employees e WHERE EXISTS (SELECT 1 FROM Projects p WHERE p.EmpID = e.EmpID);</pre>                 | Returns employees who are assigned to at least one project.                             |
| ANY / SOME                  | <pre>SELECT Name, Salary FROM Employees WHERE Salary &gt; ANY (SELECT Salary FROM Employees WHERE DeptID = 2);</pre>     | Returns employees earning more than any employee in DeptID = 2.                         |
| ALL Operator                | <pre>SELECT Name, Salary FROM Employees WHERE Salary &gt; ALL (SELECT Salary FROM Employees WHERE DeptID = 2);</pre>     | Returns employees earning more than all employees in DeptID = 2.                        |
| Scalar Subquery (in SELECT) | <pre>SELECT e.Name, (SELECT DeptName FROM Departments d WHERE d.DeptID = e.DeptID) AS Department FROM Employees e;</pre> | Returns the department name for each employee.                                          |

### 4 CORRELATED SUBQUERY EXAMPLE

```
-- Employees earning more than avg salary in their own department
SELECT e.Name, e.Salary, e.DeptID
FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary)
FROM Employees
WHERE DeptID = e.DeptID);
```

#### Remember!

- ✓ Break complex problems into smaller subqueries.
- ✓ Test subqueries separately before using.
- ✓ Use EXISTS for better performance where possible.
- ✓ Avoid using \* in large tables.

### 5 GOOD PRACTICES

- Use explicit JOINS instead of comma joins.
- Always qualify column names with table alias.
- Use subqueries only when necessary.
- Prefer EXISTS over IN for large datasets.
- Index join columns for better performance.



## 13. WINDOW FUNCTIONS (ANALYTIC FUNCTIONS)



Window Functions perform calculations across a set of rows related to the current row without collapsing the result set.

### 1 WHAT ARE WINDOW FUNCTIONS?

- Also called Analytic Functions.
- Perform calculations across a group of rows.
- Retain individual rows (do not collapse like GROUP BY).
- Use the OVER() clause.
- Syntax:  
FUNCTION() OVER (  
[PARTITION BY column]  
[ORDER BY column]  
[ROWS/RANGE frame]  
)

### 2 COMMON WINDOW FUNCTIONS

- ROW\_NUMBER() - Unique sequential number for each row.
- RANK() - Ranking with gaps.
- DENSE\_RANK() - Ranking without gaps.
- LAG() / LEAD() - Access data from previous / next row.
- FIRST\_VALUE() - First value in the window.
- LAST\_VALUE() - Last value in the window.
- NTILE(n) - Divide rows into n buckets.
- SUM(), AVG(), MIN(), MAX() - Aggregate across the window.

### Key Points



- ✓ PARTITION BY divides the data into groups.
- ✓ ORDER BY defines the order of rows in each partition.
- ✓ ROWS or RANGE (optional) defines the window frame.
- ✓ Window functions are very useful for running totals, moving averages, ranks, etc.

### 3 EXAMPLES

| Use Case                         | Example Query                                                                                                                                 | Result / Description                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| ROW_NUMBER()<br>- Unique number  | SELECT EmpID, Name, Salary,<br>ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RN<br>FROM Employees;                                              | Assigns a unique number to each row based on salary (high to low).                                 |
| RANK() vs<br>DENSE_RANK()        | SELECT Name, Salary,<br>RANK() OVER (ORDER BY Salary DESC) AS Rnk,<br>DENSE_RANK() OVER (ORDER BY Salary DESC) AS DRnk<br>FROM Employees;     | RANK() leaves gaps in ranking.<br>DENSE_RANK() does not leave gaps.<br>(e.g., 1, 2, 2, 4 for RANK) |
| LAG() - Previous<br>Row Value    | SELECT EmpID, Name, Salary,<br>LAG(Salary, 1) OVER (ORDER BY EmpID) AS Prev_Salary<br>FROM Employees;                                         | Shows the previous row's salary for each employee.                                                 |
| LEAD() - Next<br>Row Value       | SELECT EmpID, Name, Salary,<br>LEAD(Salary, 1) OVER (ORDER BY EmpID) AS Next_Salary<br>FROM Employees;                                        | Shows the next row's salary for each employee.                                                     |
| Running Total<br>(SUM OVER)      | SELECT EmpID, Name, Salary,<br>SUM(Salary) OVER (ORDER BY EmpID<br>ROWS UNBOUNDED PRECEDING) AS Running_Total<br>FROM Employees;              | Calculates running total of salary based on EmpID order.                                           |
| Moving Average<br>(3 Row Window) | SELECT EmpID, Name, Salary,<br>AVG(Salary) OVER (ORDER BY EmpID<br>ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS Moving_Avg<br>FROM Employees; | Average salary of current row, previous 1 row and next 1 row.                                      |
| Partition By<br>Example          | SELECT DeptID, EmpID, Name, Salary,<br>RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS Dept_Rank<br>FROM Employees;                 | Ranks employees within each department based on salary.                                            |
| NTILE() - Buckets                | SELECT EmpID, Name, Salary,<br>NTILE(4) OVER (ORDER BY Salary DESC) AS Salary_Quartile<br>FROM Employees;                                     | Divides employees into 4 salary quartiles.                                                         |

### 4 WINDOW FRAME CLAUSES

- ROWS UNBOUNDED PRECEDING  
- From first row of partition till current row.
- ROWS BETWEEN n PRECEDING AND n FOLLOWING  
- Defines a sliding window.
- RANGE BETWEEN ... - Based on ORDER BY value range (not row count).

If ORDER BY is not specified, whole partition is considered.



### 5 WHEN TO USE?

- ✓ Running totals and cumulative sums
- ✓ Moving averages and trends
- ✓ Rankings and leaderboards
- ✓ Comparing with previous/next rows
- ✓ Percentiles and distribution analysis
- ✓ Advanced reporting & analytics



### 6 QUICK TIPS

- ✓ Window functions keep all rows - they don't collapse data.
- ✓ Use PARTITION BY for group-wise calculations.
- ✓ Combine with WHERE / HAVING for better filtering.
- ✓ Order of execution: FROM → WHERE → GROUP BY → HAVING → WINDOW FUNCTIONS → ORDER BY.



# SQL NOTES

Slide 14

## 14. CTEs, TEMP TABLES & CASE STATEMENT



These features help to simplify complex queries, store intermediate results, and implement conditional logic.

### 1 COMMON TABLE EXPRESSIONS (CTE)

- A temporary result set that exists only within the scope of a query.
- Makes complex queries easier to read and maintain.
- Defined using WITH clause.
- Can be recursive (if supported by DBMS).

#### Example

```
WITH HighEarnings AS (  
    SELECT EmpID, Name, Salary  
    FROM Employees  
    WHERE Salary > 60000  
)  
SELECT DeptID, COUNT(*) AS Total_HighEarnings  
FROM HighEarnings  
GROUP BY DeptID;
```

#### Key Points



- ✓ CTE improves readability.
- ✓ Acts like a temporary result set.
- ✓ Exists only for the query execution.
- ✓ Can be used multiple times in the same query.

### 2 TEMPORARY TABLES

- Stored in tempdb (SQL Server) for the session.
- Useful for storing intermediate results that need to be reused.
- Can be indexed.
- Must be created and dropped manually.

#### Example

```
-- Create Temp Table  
CREATE TABLE #HighEarnings (  
    EmpID INT,  
    Name VARCHAR(100),  
    Salary DECIMAL(10,2)  
);  
-- Insert Data  
INSERT INTO #HighEarnings  
SELECT EmpID, Name, Salary  
FROM Employees  
WHERE Salary > 60000;  
-- Use Temp Table  
SELECT DeptID, COUNT(*) AS Total_HighEarnings  
FROM #HighEarnings h  
JOIN Departments d ON h.EmpID = d.EmpID  
GROUP BY DeptID;  
-- Drop Temp Table  
DROP TABLE #HighEarnings;
```

#### When to Use?



- ✓ When results are needed in multiple steps.
- ✓ When the result set is large and reused.
- ✓ When indexing the intermediate result is beneficial.

#### CTE vs Temp Table

| CTE                       | Temp Table                          |
|---------------------------|-------------------------------------|
| Exists only for the query | Exists for the session              |
| Cannot be indexed         | Can be indexed                      |
| Better for readability    | Better for performance (large data) |
| Defined in WITH clause    | Created using CREATE TABLE          |

### 3 CASE STATEMENT

- Used to return values based on conditions.
- Supports simple and searched CASE.
- Similar to IF...ELSE.

#### Simple CASE Example

```
SELECT Name, Salary,  
CASE  
    WHEN Salary >= 80000 THEN 'High'  
    WHEN Salary >= 50000 THEN 'Medium'  
    ELSE 'Low'  
END AS Salary_Category  
FROM Employees;
```

#### Searched CASE Example

```
SELECT Name, DeptID, Salary,  
CASE  
    WHEN DeptID = 1 AND Salary > 60000 THEN 'Sales-High'  
    WHEN DeptID = 2 AND Salary > 60000 THEN 'IT-High'  
    WHEN Salary >= 40000 THEN 'Normal'  
    ELSE 'Low'  
END AS Category  
FROM Employees;
```

### 4 NESTED CTE EXAMPLE



```
WITH DeptAvg AS (  
    SELECT DeptID, AVG(Salary) AS Avg_Salary  
    FROM Employees  
    GROUP BY DeptID  
)  
), HighDepts AS (  
    SELECT DeptID FROM DeptAvg  
    WHERE Avg_Salary > 50000  
)  
SELECT e.Name, e.Salary, e.DeptID  
FROM Employees e  
JOIN HighDepts h ON e.DeptID = h.DeptID  
ORDER BY e.Salary DESC;
```

### 5 BEST PRACTICES

- ✓ Use CTEs for better readability and maintainability.
- ✓ Use Temp Tables when working with large datasets or multiple processing steps.
- ✓ Drop Temp Tables when no longer needed.
- ✓ Use meaningful names for CTEs, Temp Tables and columns.
- ✓ Use CASE for clean and efficient conditional logic.



### 6 QUICK RECAP

- ★ CTE → Temporary result set within a query.
- ★ Temp Table → Stored in tempdb for the session.
- ★ CASE → Returns values based on conditions.
- ★ Choose the right tool based on performance, reusability and readability.
- ★ Practice combining these with JOINS, WINDOW FUNCTIONS and AGGREGATES for powerful queries!



Ajeet Yadav



# SQL NOTES

Slide 15

## 15. STRING FUNCTIONS, DATE FUNCTIONS & CONVERSION



These functions help to work with text, dates and convert data between different formats.

### 1 STRING FUNCTIONS

Abbr.: STR

| Function    | Description                     | Example                                   |
|-------------|---------------------------------|-------------------------------------------|
| LEN()       | Returns length of string        | LEN('SQL') → 3                            |
| UPPER()     | Converts to upper case          | UPPER('sql') → 'SQL'                      |
| LOWER()     | Converts to lower case          | LOWER('SQL') → 'sql'                      |
| SUBSTRING() | Extracts part of string         | SUBSTRING('SQLServer',1,3) → 'SQL'        |
| LEFT()      | Returns left N characters       | LEFT('Database',4) → 'Data'               |
| RIGHT()     | Returns right N characters      | RIGHT('Database',4) → 'base'              |
| REPLACE()   | Replaces a substring            | REPLACE('abcabc','a','x') → 'xabcx'       |
| TRIM()      | Removes leading/trailing spaces | TRIM(' SQL ') → 'SQL'                     |
| CONCAT()    | Concatenates strings            | CONCAT('SQL', ' ', 'Notes') → 'SQL Notes' |

### 2 DATE FUNCTIONS

Abbr.: DT

| Function     | Description                 | Example                                     |
|--------------|-----------------------------|---------------------------------------------|
| GETDATE()    | Returns current date & time | GETDATE() → '2025-06-03 ...'                |
| CURRENT_DATE | Returns current date        | CURRENT_DATE → '2025-06-03'                 |
| DATEADD()    | Adds date/time interval     | DATEADD(DAY,5,'2025-06-03') → '2025-06-08'  |
| DATEDIFF()   | Difference between dates    | DATEDIFF(DAY,'2025-06-01','2025-06-03') → 2 |
| YEAR()       | Extracts year               | YEAR(GETDATE()) → 2025                      |
| MONTH()      | Extracts month              | MONTH(GETDATE()) → 6                        |
| DAY()        | Extracts day                | DAY(GETDATE()) → 3                          |
| EOMONTH()    | Last day of month           | EOMONTH('2025-06-03') → '2025-06-30'        |

### 3 CONVERSION FUNCTIONS

Abbr.: CVT

Used to convert data from one data type to another.

| Function      | Description                        | Syntax                                   | Example                                         |
|---------------|------------------------------------|------------------------------------------|-------------------------------------------------|
| CAST()        | Converts to specified data type    | CAST(expression AS datatype)             | CAST('123' AS INT) → 123                        |
| CONVERT()     | Converts with style (for dates)    | CONVERT(datatype, expression, style)     | CONVERT(VARCHAR, GETDATE(), 103) → '03/06/2025' |
| TRY_CAST()    | Safe cast, returns NULL on failure | TRY_CAST(expression AS datatype)         | TRY_CAST('abc' AS INT) → NULL                   |
| TRY_CONVERT() | Safe convert with style            | TRY_CONVERT(datatype, expression, style) | TRY_CONVERT(INT, 'abc') → NULL                  |

### 4 DATE FORMAT STYLES (COMMON)

| Style | Format              | Example (CONVERT(VARCHAR, GETDATE(), style)) |
|-------|---------------------|----------------------------------------------|
| 101   | mm/dd/yyyy          | 06/03/2025                                   |
| 103   | dd/mm/yyyy          | 03/06/2025                                   |
| 104   | dd.mm.yyyy          | 03.06.2025                                   |
| 105   | dd-mm-yyyy          | 03-06-2025                                   |
| 120   | yyyy-mm-dd hh:mi:ss | 2025-06-03 14:30:00                          |
| 126   | yyyy-mm-ddThh:mi:ss | 2025-06-03T14:30:00                          |

### 5 QUICK EXAMPLES

- Length of employee name  
SELECT Name, LEN(Name) AS NameLength FROM Employees;
- Uppercase email  
SELECT Email, UPPER(Email) AS EmailUpper FROM Employees;
- Extract first 3 characters of Department  
SELECT Department, LEFT(Department,3) AS DeptCode FROM Employees;
- Employees hired in last 7 days  
SELECT \* FROM Employees  
WHERE HireDate >= DATEADD(DAY,-7, GETDATE());
- Convert salary to varchar  
SELECT CAST(Salary AS VARCHAR(20)) AS SalaryText FROM Employees;
- Format date as dd/mm/yyyy  
SELECT CONVERT(VARCHAR, HireDate, 103) AS HireDateDMY FROM Employees;

### 6 TIPS & REMEMBER



- ✓ Use LEN() to find length of text.
- ✓ TRIM() helps to clean extra spaces.
- ✓ DATEADD() and DATEDIFF() are very powerful for date calculations.
- ✓ Use TRY\_CAST() / TRY\_CONVERT() to avoid errors in conversions.
- ✓ Choose correct style in CONVERT() for date formatting.
- ✓ Window functions + Date functions = Powerful analytics!

### At a Glance



- ★ String Functions → Work with text data
- ★ Date Functions → Extract, add, compare dates
- ★ Conversion Functions → Change data types
- ★ CTEs, Temp Tables, CASE → Simplify complex logic

Practice more, write better queries!



Ajeet Yadav